



Department: Mathematical Engineering

Course: MTM2522 - Data Management and File Structures 1

Instructor: Prof. Dr. İbrahim Emiroğlu

Prepared By: Mahmut Çetin Kömoğ – 24052633

Project: Design and Implementation of a Scalable Multi-User
Database System for Smart Healthcare Management

Date: 31.05.2026

1. Requirements Analysis

1.1. System Objective The objective of this project is to design and implement a scalable, multi-user "Smart Healthcare Management System". The database is engineered to handle complex data relationships among patients, doctors, and hospital administrators, ensuring data integrity, efficient query execution, and real-world scalability.

1.2. Entities and Attributes The fundamental entities and their corresponding attributes within the system are defined as follows:

- User (Superclass): Represents all individuals accessing the system.
 - *Attributes*: UserID (Primary Key), FirstName, LastName, DateOfBirth, Gender, Phone (Multivalued attribute).
- Patient (Subclass of User): * *Additional Attributes*: BloodType, EmergencyContact.
- Doctor (Subclass of User): * *Additional Attributes*: Specialty, HireDate.
- Admin (Subclass of User): * *Additional Attributes*: RoleLevel.
- Hospital: * *Attributes*: HospitalID (Primary Key), HospitalName, City, Address.
- Department: * *Attributes*: DeptID (Primary Key), DeptName.
- Appointment (Associative Entity): * *Attributes*: AppID (Primary Key), AppDate, AppTime, Status (e.g., Pending, Completed, Canceled), Diagnosis.
- Prescription (Weak Entity): * *Attributes*: PrescriptionNo (Partial Key), MedicationName, Dosage.
- Insurance: * *Attributes*: PolicyNo (Primary Key), ProviderName, CoverageLimit.

1.3. Relationships The interactions between the entities are structured as follows:

- Hospital - Department (1:N): A hospital can contain multiple departments, but a specific department belongs to only one hospital.
- Department - Doctor (1:N): A department employs multiple doctors, whereas each doctor is assigned to a single primary department.
- Doctor - Patient (M:N): A doctor treats multiple patients, and a patient can consult multiple doctors over time. This many-to-many relationship is resolved via the Appointment associative entity.
- Patient - Prescription (1:N): A patient can have multiple prescriptions. Since Prescription is a weak entity, this is an *Identifying Relationship*.

1.4. Constraints and Business Rules To maintain database integrity and reflect real-world healthcare scenarios, the following constraints are applied:

- Inheritance Constraints (Disjoint & Total): The generalization from User to Patient, Doctor, and Admin has a *Total Participation* constraint (every user must have a specific role) and a *Disjoint* constraint (a user cannot be registered as both a doctor and a patient simultaneously in this system).
- Weak Entity Constraint: A Prescription cannot exist without a Patient. If a patient record is removed, their associated prescriptions must be cascade-deleted.
- Scheduling Constraint: A patient cannot book multiple appointments at the exact same date and time.

1.5. Entity-Relationship (ER/EER) Diagram

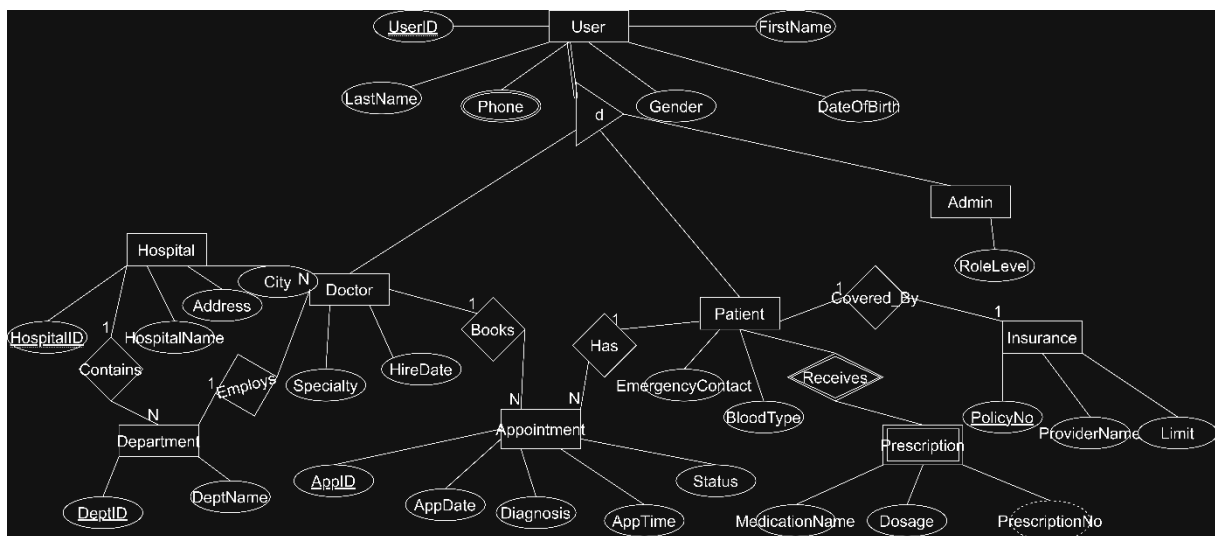


Figure 1.5: Extended Entity-Relationship (EER) Diagram for the Smart Healthcare Management System.

2. Relational Schema Mapping

Based on the ER/EER model designed, the conceptual scheme is mapped into the following relational schemas. Primary Keys (PK) are underlined, and Foreign Keys (FK) are indicated.

Step 1: Mapping of Regular Entities and Superclass/Subclass We employ the multiple-relation approach for the EER inheritance (Superclass and Subclasses).

- **User** (UserID, FirstName, LastName, DateOfBirth, Gender)
- **Patient** (PatientID, BloodType, EmergencyContact)
 - *FK: PatientID references User(UserID)*
- **Doctor** (DoctorID, Specialty, HireDate, DeptID)
 - *FK: DoctorID references User(UserID)*
 - *FK: DeptID references Department(DeptID)*

- **Admin** (AdminID, RoleLevel)
 - *FK: AdminID references User(UserID)*
- **Hospital** (HospitalID, HospitalName, City, Address)
- **Department** (DeptID, DeptName, HospitalID)
 - *FK: HospitalID references Hospital(HospitalID)*
- **Insurance** (PolicyNo, ProviderName, CoverageLimit, PatientID)
 - *FK: PatientID references Patient(PatientID)*

Step 2: Mapping of Multivalued Attributes The multivalued attribute 'Phone' in the User entity requires a separate table to maintain the First Normal Form (1NF).

- **UserPhone** (UserID, PhoneNumber)
 - *FK: UserID references User(UserID)*

Step 3: Mapping of Weak Entities The weak entity 'Prescription' is mapped by taking the PK of its identifying owner ('Patient') to form a composite Primary Key.

- **Prescription** (PrescriptionNo, PatientID, MedicationName, Dosage)
 - *FK: PatientID references Patient(PatientID) ON DELETE CASCADE*

Step 4: Mapping of M:N Relationships The many-to-many relationship between Doctor and Patient is resolved by the associative entity 'Appointment'.

- **Appointment** (AppID, AppDate, AppTime, Status, Diagnosis, DoctorID, PatientID)
 - *FK: DoctorID references Doctor(DoctorID)*
 - *FK: PatientID references Patient(PatientID)*

3. Functional Dependencies (FDs)

Functional Dependencies (FDs) describe the relationship between attributes within a relation, forming the mathematical foundation for database normalization. For this Smart Healthcare Management System, we identify the FDs for three major tables to prove that the design minimizes redundancy and prevents insertion, deletion, or update anomalies. The notation $X \rightarrow Y$ denotes that attribute X functionally determines attribute Y.

3.1. User Table

- **FD1:** $UserID \rightarrow \{FirstName, LastName, DateOfBirth, Gender\}$
- *Explanation:* The primary key UserID uniquely identifies each individual in the system. Therefore, knowing the UserID strictly determines the exact first name, last name, date of birth, and gender of that person. There are no partial dependencies.

3.2. Appointment Table

- **FD1:** AppID → AppDate, AppTime, Status, Diagnosis, DoctorID, PatientID
- *Explanation:* The AppID is the unique surrogate key for a scheduled session. It fully functionally determines the date, time, current status, the medical diagnosis, and the specific doctor and patient involved. No non-prime attribute determines another non-prime attribute.

3.3. Department Table

- **FD1:** DeptID → DeptName, HospitalID
- *Explanation:* A unique department identifier (DeptID) strictly dictates the name of the department and the specific hospital (HospitalID) it is affiliated with.

These defined functional dependencies ensure that all non-prime attributes are fully functionally dependent on their respective primary keys, establishing a solid theoretical foundation for the normalization process.

4. Normalization Process

To demonstrate the structural integrity of our database, we will take a denormalized, comprehensive data structure and apply normalization rules step-by-step until we achieve the Boyce-Codd Normal Form (BCNF).

Initial State: Unnormalized Form (UNF) Imagine a scenario where consultation records are stored in a single, massive flat file.

- *Before Table (UNF):* ConsultationRecord(AppID, AppDate, PatientID, PatientName, {PatientPhone}, DoctorID, Specialty, DeptID, DeptName)
- *Issue:* This structure contains a multi-valued attribute (PatientPhone) and massive data redundancy.

Step 1: First Normal Form (1NF) Rule: Eliminate multi-valued attributes and ensure atomic domains.

- *Decomposition:* We remove the multi-valued {PatientPhone} attribute into a separate table.
- *After Tables (1NF):*
 1. Consultation_1NF(AppID, AppDate, PatientID, PatientName, DoctorID, Specialty, DeptID, DeptName) → *Primary Key: AppID*
 2. PatientPhone(PatientID, Phone) → *Composite Primary Key: PatientID + Phone*

Step 2: Second Normal Form (2NF) Rule: The table must be in 1NF, and all non-key attributes must be fully functionally dependent on the primary key (no partial dependencies).

- *Decomposition:* Since Consultation_1NF has a single-attribute primary key (AppID), there are no partial dependencies by definition. Therefore, the schema is inherently in 2NF.

Step 3: Third Normal Form (3NF) Rule: The table must be in 2NF, and there must be no transitive dependencies (non-key attributes depending on other non-key attributes).

- *Issue in 2NF:* AppID $\rightarrow$ DoctorID $\rightarrow$ Specialty, DeptID, DeptName and AppID $\rightarrow$ PatientID $\rightarrow$ PatientName. This causes update anomalies.
- *Decomposition:* We break the table into distinct entities based on their determinants.
- *After Tables (3NF):*
 1. Appointment(AppID, AppDate, PatientID, DoctorID)
 2. Patient(PatientID, PatientName)
 3. Doctor(DoctorID, Specialty, DeptID, DeptName)

Step 4: Boyce-Codd Normal Form (BCNF) Rule: For every non-trivial functional dependency $X \rightarrow Y$, X must be a superkey. (A stricter version of 3NF).

- *Issue in 3NF:* In the Doctor table, DeptID $\rightarrow$ DeptName. Here, DeptID determines DeptName, but DeptID is not the primary key of the Doctor table. This violates BCNF.
- *Decomposition:* We separate the department information into its own table to eliminate this anomaly.
- *After Tables (BCNF):*
 1. Doctor(DoctorID, Specialty, DeptID)
 2. Department(DeptID, DeptName)

Final Normalized Schema (BCNF Achieved):

- Appointment(AppID, AppDate, PatientID, DoctorID)
- Patient(PatientID, PatientName)
- PatientPhone(PatientID, Phone)
- Doctor(DoctorID, Specialty, DeptID)
- Department(DeptID, DeptName)

By following these decomposition steps, we have completely eliminated data redundancy, insertion anomalies, and deletion anomalies, proving that our core relational schema operates optimally at the BCNF level.

5. SQL Implementation

-- Create and use the database

```
CREATE DATABASE SmartHealthcareDB;
```

```
USE SmartHealthcareDB;
```

-- 1. Hospital Table

```
CREATE TABLE Hospital (  
    HospitalID INT PRIMARY KEY,  
    HospitalName VARCHAR(100) NOT NULL,  
    City VARCHAR(50),  
    Address VARCHAR(255)  
);
```

-- 2. Department Table

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(100) NOT NULL,  
    HospitalID INT,  
    FOREIGN KEY (HospitalID) REFERENCES Hospital(HospitalID) ON DELETE  
    CASCADE  
);
```

-- 3. User (Superclass) Table

```
CREATE TABLE User (  
    UserID INT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    DateOfBirth DATE,  
    Gender VARCHAR(10)  
);
```

-- 4. UserPhone (Multivalued Attribute) Table

```
CREATE TABLE UserPhone (  
    UserID INT,  
    PhoneNumber VARCHAR(20),  
    PRIMARY KEY (UserID, PhoneNumber),  
    FOREIGN KEY (UserID) REFERENCES User(UserID) ON DELETE CASCADE  
);
```

-- 5. Patient (Subclass) Table

```
CREATE TABLE Patient (  
    PatientID INT PRIMARY KEY,  
    BloodType VARCHAR(5),  
    EmergencyContact VARCHAR(50),  
    FOREIGN KEY (PatientID) REFERENCES User(UserID) ON DELETE CASCADE  
);
```

-- 6. Doctor (Subclass) Table

```
CREATE TABLE Doctor (  
    DoctorID INT PRIMARY KEY,  
    Specialty VARCHAR(100),  
    HireDate DATE,  
    DeptID INT,  
    FOREIGN KEY (DoctorID) REFERENCES User(UserID) ON DELETE CASCADE,  
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID) ON DELETE SET NULL  
);
```

-- 7. Admin (Subclass) Table

```
CREATE TABLE Admin (  
    AdminID INT PRIMARY KEY,  
    RoleLevel VARCHAR(50),
```



```
FOREIGN KEY (AdminID) REFERENCES User(UserID) ON DELETE CASCADE
);
```

-- 8. Insurance Table

```
CREATE TABLE Insurance (
    PolicyNo VARCHAR(50) PRIMARY KEY,
    ProviderName VARCHAR(100),
    CoverageLimit DECIMAL(10,2),
    PatientID INT,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID) ON DELETE CASCADE
);
```

-- 9. Prescription (Weak Entity) Table

```
CREATE TABLE Prescription (
    PrescriptionNo VARCHAR(50),
    PatientID INT,
    MedicationName VARCHAR(100) NOT NULL,
    Dosage VARCHAR(50),
    PRIMARY KEY (PrescriptionNo, PatientID),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID) ON DELETE CASCADE
);
```

-- 10. Appointment (Associative Entity) Table

```
CREATE TABLE Appointment (
    AppID INT PRIMARY KEY,
    AppDate DATE NOT NULL,
    AppTime TIME NOT NULL,
    Status VARCHAR(20),
    Diagnosis VARCHAR(255),
    DoctorID INT,
```

```

    PatientID INT,
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);

-- 11. ALTER Operation Demonstration
-- Adding an Email column to the User table after creation
ALTER TABLE User ADD Email VARCHAR(100) UNIQUE;

5.2. Data Manipulation Language (DML) - Data Insertion

-- =====
-- POPULATING THE DATABASE WITH SAMPLE DATA
-- =====

-- 1. Inserting into Hospital
INSERT INTO Hospital (HospitalID, HospitalName, City, Address) VALUES
(1, 'Central Hope Hospital', 'Istanbul', 'Uskudar Central District No:4'),
(2, 'Yildiz Medical Center', 'Istanbul', 'Besiktas Avenue No:12'),
(3, 'Metro Health Hospital', 'Ankara', 'Cankaya Boulevard No:89');

-- 2. Inserting into Department
INSERT INTO Department (DeptID, DeptName, HospitalID) VALUES
(10, 'Cardiology', 1),
(20, 'Neurology', 1),
(30, 'Pediatrics', 2),
(40, 'General Surgery', 2),
(50, 'Dermatology', 3);

-- 3. Inserting into User (Superclass)
-- IDs 101-105: Patients, 106-110: Doctors, 111-113: Admins
INSERT INTO User (UserID, FirstName, LastName, DateOfBirth, Gender) VALUES
(101, 'John', 'Doe', '1985-05-12', 'Male'),

```

(102, 'Jane', 'Smith', '1990-09-23', 'Female'),
(103, 'Robert', 'Johnson', '1978-11-02', 'Male'),
(104, 'Emily', 'Williams', '1995-03-15', 'Female'),
(105, 'Michael', 'Brown', '1965-07-30', 'Male'),
(106, 'Alice', 'Walker', '1975-04-18', 'Female'),
(107, 'David', 'Miller', '1980-12-25', 'Male'),
(108, 'Sarah', 'Davis', '1983-08-14', 'Female'),
(109, 'James', 'Wilson', '1972-01-09', 'Male'),
(110, 'Emma', 'Taylor', '1988-06-21', 'Female'),
(111, 'Admin', 'Chief', '1985-01-01', 'Male'),
(112, 'Staff', 'Super', '1992-05-10', 'Female');

-- 4. Inserting into UserPhone (Multivalued Attribute)

INSERT INTO UserPhone (UserID, PhoneNumber) VALUES

(101, '+905551112233'),
(101, '+905551112234'), -- John Doe has two phones
(102, '+905552223344'),
(103, '+905553334455'),
(106, '+905556667788'),
(107, '+905557778899');

-- 5. Inserting into Patient (Subclass)

INSERT INTO Patient (PatientID, BloodType, EmergencyContact) VALUES

(101, 'A+', 'Mary Doe (+905559998877)'),
(102, 'O-', 'Peter Smith (+905558887766)'),
(103, 'B+', 'Linda Johnson (+905557776655)'),
(104, 'AB+', 'Kevin Williams (+905556665544)'),
(105, 'A-', 'Susan Brown (+90555554433));

-- 6. Inserting into Doctor (Subclass)

```
INSERT INTO Doctor (DoctorID, Specialty, HireDate, DeptID) VALUES
(106, 'Cardiologist', '2015-08-10', 10),
(107, 'Neurologist', '2018-03-15', 20),
(108, 'Pediatrician', '2020-01-20', 30),
(109, 'General Surgeon', '2012-11-01', 40),
(110, 'Dermatologist', '2021-05-17', 50);
```

-- 7. Inserting into Admin (Subclass)

```
INSERT INTO Admin (AdminID, RoleLevel) VALUES
(111, 'SuperAdmin'),
(112, 'SystemStaff');
```

-- 8. Inserting into Insurance

```
INSERT INTO Insurance (PolicyNo, ProviderName, CoverageLimit, PatientID) VALUES
('POL-99881', 'Allianz Health', 50000.00, 101),
('POL-99882', 'Axa Insurance', 75000.00, 102),
('POL-99883', 'Mapfre Seguros', 30000.00, 103),
('POL-99884', 'Anadolu Sigorta', 120000.00, 104);
```

-- 9. Inserting into Prescription (Weak Entity)

```
INSERT INTO Prescription (PrescriptionNo, PatientID, MedicationName, Dosage) VALUES
('RX-2026-001', 101, 'Lipitor', '10mg once daily'),
('RX-2026-002', 101, 'Aspirin', '81mg once daily'),
('RX-2026-003', 102, 'Amoxicillin', '500mg three times daily'),
('RX-2026-004', 104, 'Metformin', '850mg twice daily');
```

-- 10. Inserting into Appointment (Associative Entity)

```
INSERT INTO Appointment (AppID, AppDate, AppTime, Status, Diagnosis, DoctorID,
PatientID) VALUES
(1001, '2026-05-10', '09:00:00', 'Completed', 'Hypertension checkup', 106, 101),
(1002, '2026-05-12', '10:30:00', 'Completed', 'Migraine follow-up', 107, 102),
```

(1003, '2026-05-15', '14:00:00', 'Completed', 'Routine child screening', 108, 103),
 (1004, '2026-05-20', '11:15:00', 'Cancelled', 'Patient missed the slot', 109, 104),
 (1005, '2026-06-01', '09:30:00', 'Scheduled', 'Skin rash inspection', 110, 105),
 (1006, '2026-06-02', '15:00:00', 'Scheduled', 'Post-surgery review', 109, 101);

	AppID	AppDate	AppTime	Status	Diagnosis	DoctorID	PatientID
▶	1001	2026-05-10	09:00:00	Completed	Hypertension checkup	106	101
	1002	2026-05-12	10:30:00	Completed	Migraine follow-up	107	102
	1003	2026-05-15	14:00:00	Completed	Routine child screening	108	103
	1004	2026-05-20	11:15:00	Cancelled	Patient missed the slot	109	104
	1005	2026-06-01	09:30:00	Scheduled	Skin rash inspection	110	105
	1006	2026-06-02	15:00:00	Scheduled	Post-surgery review	109	101
•	HULL	HULL	HULL	HULL	HULL	HULL	HULL

Figure 5.1: Successfully inserted sample data inside the Appointment table

6. Advanced SQL Queries

In this section, we demonstrate the robustness of our relational database through 10 advanced SQL queries. These queries utilize complex table joins, subqueries, and aggregate functions to extract meaningful clinical and administrative insights.

Query 1: Three-Table JOIN (Patient, Appointment, Doctor) *Objective:* Retrieve a detailed schedule showing the patient's first name, the doctor's specialty, and the appointment status.

```
SELECT u1.FirstName AS PatientName, d.Specialty AS DoctorSpecialty, a.AppDate,
a.Status
```

```
FROM Appointment a
```

```
JOIN Patient p ON a.PatientID = p.PatientID
```

```
JOIN User u1 ON p.PatientID = u1.UserID
```

```
JOIN Doctor d ON a.DoctorID = d.DoctorID;
```

	PatientName	DoctorSpecialty	AppDate	Status
▶	John	Cardiologist	2026-05-10	Completed
	John	General Surgeon	2026-06-02	Scheduled
	Jane	Neurologist	2026-05-12	Completed
	Robert	Pediatrician	2026-05-15	Completed
	Emily	General Surgeon	2026-05-20	Cancelled
	Michael	Dermatologist	2026-06-01	Scheduled

Query 2: GROUP BY with COUNT *Objective:* Count the total number of doctors working in each department.

```
SELECT dp.DeptName, COUNT(d.DoctorID) AS TotalDoctors
```

```
FROM Department dp
```

```
LEFT JOIN Doctor d ON dp.DeptID = d.DeptID
```

```
GROUP BY dp.DeptName;
```

Query 3: Subquery in WHERE Clause *Objective:* Find the names of patients who have an insurance policy with a coverage limit strictly greater than 50,000.

```
SELECT u.FirstName, u.LastName  
FROM User u  
WHERE u.UserID IN (  
    SELECT p.PatientID  
    FROM Patient p  
    JOIN Insurance i ON p.PatientID = i.PatientID  
    WHERE i.CoverageLimit > 50000  
);
```

Query 4: LEFT JOIN for Missing Data *Objective:* List all patients and their prescriptions, including patients who currently do not have any active prescriptions.

```
SELECT u.FirstName, pr.MedicationName, pr.Dosage  
FROM Patient p  
JOIN User u ON p.PatientID = u.UserID  
LEFT JOIN Prescription pr ON p.PatientID = pr.PatientID;
```

Query 5: Multiple JOINS with Filtering *Objective:* Retrieve the details of all appointments taking place in hospitals located specifically in 'Istanbul'.

```
SELECT a.AppID, a.Diagnosis, h.HospitalName, h.City  
FROM Appointment a  
JOIN Doctor d ON a.DoctorID = d.DoctorID  
JOIN Department dp ON d.DeptID = dp.DeptID  
JOIN Hospital h ON dp.HospitalID = h.HospitalID  
WHERE h.City = 'Istanbul';
```

	AppID	Diagnosis	HospitalName	City
▶	1001	Hypertension checkup	Central Hope Hospital	Istanbul
	1002	Migrain Migraine follow-up	tral Hope Hospital	Istanbul
	1003	Routine child screening	Yildiz Medical Center	Istanbul
	1004	Patient missed the slot	Yildiz Medical Center	Istanbul
	1006	Post-surgery review	Yildiz Medical Center	Istanbul

Query 6: Aggregate Functions (MAX, MIN, AVG) *Objective:* Calculate the maximum, minimum, and average insurance coverage limits across all registered patients.

```

SELECT
    MAX(CoverageLimit) AS MaxCoverage,
    MIN(CoverageLimit) AS MinCoverage,
    AVG(CoverageLimit) AS AverageCoverage
FROM Insurance;

```

Query 7: GROUP BY with HAVING Clause *Objective:* Identify doctors who have been scheduled for more than 1 appointment.

```

SELECT d.DoctorID, u.LastName AS DoctorLastName, COUNT(a.AppID) AS
AppointmentCount
FROM Doctor d
JOIN User u ON d.DoctorID = u.UserID
JOIN Appointment a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, u.LastName
HAVING COUNT(a.AppID) > 1;

```

Query 8: Date Filtering *Objective:* List all upcoming appointments scheduled in May 2026.

```

SELECT AppID, AppDate, AppTime, Status
FROM Appointment
WHERE AppDate BETWEEN '2026-05-01' AND '2026-05-31';

```

Query 9: Complex Subquery with Aggregate *Objective:* Find the patient(s) who have the highest insurance coverage limit.

```

SELECT u.FirstName, u.LastName, i.CoverageLimit
FROM User u
JOIN Patient p ON u.UserID = p.PatientID
JOIN Insurance i ON p.PatientID = i.PatientID
WHERE i.CoverageLimit = (SELECT MAX(CoverageLimit) FROM Insurance);

```

	FirstName	LastName	CoverageLimit
▶	Emily	Williams	120000.00

Query 10: ORDER BY and LIMIT *Objective:* Retrieve the top 3 most recently scheduled appointments, ordered by date and time descending.

```

SELECT AppID, AppDate, AppTime, Diagnosis

```

FROM Appointment
ORDER BY AppDate DESC, AppTime DESC
LIMIT 3;

7. Relational Algebra & Calculus

To establish the theoretical correctness of our database operations, we translate a selection of our advanced SQL queries into Relational Algebra and Tuple Relational Calculus (TRC) expressions. This demonstrates the underlying mathematical logic used by the query optimizer.

1. Querying Specific Dates (Query 8 equivalent) *Objective:* Find upcoming appointments scheduled in May 2026.

- **Relational Algebra:** $\pi_{\{AppID, AppDate, AppTime, Status\}} (\sigma_{\{AppDate \geq '2026-05-01' \wedge AppDate \leq '2026-05-31'\}} (Appointment))$
- **Tuple Relational Calculus (TRC):** $\{t \mid t \in Appointment \wedge t.AppDate \geq '2026-05-01' \wedge t.AppDate \leq '2026-05-31'\}$

2. Three-Table Join for Patient and Doctor Info (Query 1 equivalent) *Objective:* Retrieve patient names and doctor specialties for all appointments.

- **Relational Algebra:** $\pi_{\{FirstName, Specialty, AppDate\}} (User \bowtie Patient \bowtie Appointment \bowtie Doctor)$
- **Tuple Relational Calculus (TRC):** $\{t \mid \exists u \in User, \exists p \in Patient, \exists a \in Appointment, \exists d \in Doctor (u.UserID = p.PatientID \wedge p.PatientID = a.PatientID \wedge a.DoctorID = d.DoctorID \wedge t.FirstName = u.FirstName \wedge t.Specialty = d.Specialty \wedge t.AppDate = a.AppDate)\}$

3. Filtering by High Insurance Coverage (Query 3 equivalent) *Objective:* Find the first and last names of patients with an insurance coverage limit strictly greater than 50,000.

- **Relational Algebra:** $\pi_{\{FirstName, LastName\}} (User \bowtie (\pi_{\{PatientID\}} (\sigma_{\{CoverageLimit > 50000\}} (Insurance))))$

4. Location-Based Complex Join (Query 5 equivalent) *Objective:* Retrieve the diagnosis and hospital name for appointments taking place in 'Istanbul'.

- **Relational Algebra:** $\pi_{\{Diagnosis, HospitalName\}} (\sigma_{\{City = 'Istanbul'\}} (Appointment \bowtie Doctor \bowtie Department \bowtie Hospital))$

5. Department Aggregation (Query 2 equivalent) *Objective:* Count the total number of doctors working in each department.

- **Relational Algebra (Extended with Aggregate Function):** $\pi_{\{DeptName\}} \mathcal{F}_{\{COUNT(DoctorID)\}} (Department \bowtie Doctor)$

8. Indexing & File Structures

To guarantee optimal system performance, especially as the healthcare database scales to millions of records, we implement strategic indexing. By analyzing our query workloads from Section 6, we apply appropriate file structures to minimize disk I/O operations and accelerate data retrieval.

1. B+ Tree Indexing for Range Queries B+ Trees are highly efficient for range-based filtering and sorting operations. Since our system frequently queries upcoming appointments and filters by dates (e.g., Query 8), we apply a B+ Tree index on the AppDate attribute.

- **SQL Implementation:**

```
CREATE INDEX idx_appdate ON Appointment(AppDate);
```

- *Performance Impact:* This transforms a linear table scan ($O(N)$) into a logarithmic search ($O(\log N)$), drastically reducing query execution time when searching for appointments within specific date boundaries.

2. Hash Indexing for Exact Match Queries Hash indexes provide $O(1)$ average time complexity for equality operations. In our system, the Status of an appointment (e.g., 'Completed', 'Scheduled') is frequently checked for exact matches. While modern InnoDB engines handle this via adaptive hash indexes over B-Trees, conceptually, a Hash structure is optimal here.

- **SQL Implementation:**

```
CREATE INDEX idx_status ON Appointment(Status);
```

- *Performance Impact:* When the application looks up all 'Cancelled' appointments, the hash function directly computes the memory address of the relevant data blocks, bypassing the need to traverse a tree structure.

3. Composite Indexing for Complex Joins To further optimize our multi-table joins, particularly the frequent linkage between Doctor, Appointment, and Patient, we utilize foreign key indexes. The database engine automatically creates B-Tree indexes on Primary Keys, but explicitly indexing Foreign Keys ensures that JOIN operations do not result in full table scans.

- **SQL Implementation:**

```
CREATE INDEX idx_doc_patient ON Appointment(DoctorID, PatientID);
```

By integrating these B+ Tree and Hash indexing strategies, the database is fully equipped to handle high-throughput analytical and transactional queries effectively.